

Introduction

Welcome to the fundamental UniVerse application server course! In this course, you will learn the essential concepts and skills related to the UniVerse application server and how it is used to store, organize, and manage data.

How to get started in this course:

1. Have docker or one if it's many variants loaded on the computer where you want to run the docker image:

Most versions of Linux will have either podman or docker. If not present, use your package manager and load whichever loads.

Windows will run docker images with rancher desktop. The desktop interface is a bit klunky for docker, but you can go to powershell and run docker as if you were on Linux.

There are also versions available for MacOS.

2. Create a folder/directory for your MVTUTOR account and lesson plan. You will want this to be an empty folder/directory.

It could be /savedUVE or home/me/savedUVE or C:\users\del\ savedUVE or any other directory where MVTUTOR can be placed.

3. Load the universe-essentials.tar image into your docker image store

```
docker load -i universe-essentials.tar
```

4. Run the command to start docker/podman (/savedUVE before the colon would be directory from step 2)

```
docker run -it --rm -v /savedUVE:/myuv:z universe-essentials:latest
```

This opens the Terminal and connects to the application server.

The port allocation will allow docker to listen on 32438 for VSCODE to work

5. Your version of MVTUTOR and this class will be in the /savedUVE or other directory specified in step 2.
6. If you ever want to reset to the original simply move or delete the MVTUTOR in the folder/directory from step 2 and it will be recreated next time you run the container.

1.1 Querying files and records

You are now logged into your MultiValue Application Server. Let's interact with the data inside it!

1. Enter the following to see a list of customers.

```
SORT CUSTOMERS
```

Ensure you type this sentence in UPPERCASE. Sentences and statements in lowercase are not recognized by the query language.

2. Press any key on your keyboard to see the next page.

3. Press **Q** to quit the listing.
4. Enter the sentence below to see if life gives us lemons.

```
LIST PRODUCTS WITH NAME = "Lemons"
```

Note that the actual **NAME** field in the **PRODUCTS** file contains uppercase L and the rest lowercase.

1.2 Understanding Terminology

Watch Dan explain important terminology.

Click this link to open a video hosted in the course.

[Understanding Terminology](#)

1.3 Using VERBS and building Sentences

Now that you know a bit more about how data is organized inside a MultiValue Database, let's look at how to retrieve data using MultiValue's query language.

Do you remember how we checked if life gives us lemons? To do so, we used this sentence:

```
LIST PRODUCTS WITH NAME = "Lemons"
```

All **sentences** begin with a **verb** (an action word) followed by a **noun** (a file name), for example:

Verb (command) **Noun** (file name)

```
LIST PRODUCTS
```

LIST PRODUCTS does something (**lists**) a particular file (**products**). To find out if that list of products contains lemons, we can add criteria to the sentence. But let's focus on some simple verbs and nouns for now:

1. Enter the sentence to see a list of customers sorted by their primary key.

```
SORT CUSTOMERS
```

2. Enter the sentence to see the same list of customers in their storage order.

```
LIST CUSTOMERS
```

Each customer is listed with its fields, for example: Customer Name, Store Name, and Address, and their associated values.

You may notice that LIST CUSTOMERS is similar to SORT CUSTOMERS that we used when Exploring the MVTUTOR account. SORT is the same as LIST, but if no SORT criteria are specified, the data is sorted by the primary key, in this case, CUSTID (more about primary keys later).

3. To see a list of products, enter the sentence.

```
LIST PRODUCTS
```

The *PRODUCTS* file has its own fields and data records, including **Product Number**, **Class**, **Price**, and **Weight**.

4. Now let's see how many records we have for ORDERS, by entering a count command.

```
COUNT ORDERS
```

Rather than showing you individual records, you just get a count of the number of records in the ORDERS file.

5. To see how many clients we have, enter the appropriate count command.

```
COUNT CLIENTS
```

6. What if we wanted to see the raw content of a file without any report formatting?

```
LIST.ITEM ORDERS
```

This lists the raw content in the ORDERS file.

Notice you don't see any field names, only primary keys and record data. Multi-valued fields are separated by a value mark (VM) which is displayed as a ² in the display.

As with other languages, there are some verbs that can stand by themselves without any nouns, such as:

- **LISTF** : Lists all files available to this account
- **LISTFL** : Lists only files local to this account
- **LISTV** : Lists all the verbs in the account you are logged into
- **LISTU** : List users on the system
- **L0** : Logs out of the account

There are many more verbs you can use to retrieve and manipulate data in your account. We'll cover more common verbs through this course. If you want to start exploring more, see the documentation for a full list of commands.

1.3 Constructing sentences with selection criteria and display criteria to list and sort data

Like other languages, MultiValue's query language has grammar rules that apply when using sentences to interact with data. Building from a simple sentence using VERBS and NOUNS, you can create more complicated sentences to refine your search query. You can add:

- **Selection criteria** to specify which items in the file you want to see.
- **Sort criteria** to specify how you want to order the items.
- **Display criteria** to specify which data will be displayed.
- **Modifiers** to change the output.

You can then build a sentence like this:

```
LIST PRODUCTS WITH DESC LIKE "...berries..." BY DESC PRODID DESC QOH ID-SUPP
```

This sentence lists products that have "berries" somewhere in the description, sorts the results by description, and displays the product number (PRODID), description (DESC), and quantity on hand (QOH). The output is modified to suppress the product ID (the primary key for the PRODUCTS file).

Let's put this into practice to list and sort the data in our MVTUTOR account:

1. Previously you listed the products which showed you all the data fields associated with each product. Now let's use display criteria to change what we see. Enter:

```
LIST PRODUCTS CLASS DESC PRICE
```

This shows you a list of products, displaying the Product Number (the primary key), the class, description, and price.

Use **Q** to quit the list or press **Enter** for next page

2. To know which fields you can use as display criteria for your PRODUCTS file, enter:

```
LIST DICT PRODUCTS
```

DICT stands for Dictionary. We'll discuss the dictionary later. For now, what is important are the field names that correspond to the column headings you can use to select your display criteria.

The field PRODID is listed as Field Number 0 and is a synonym for the @ID field, which is the default field name for the primary key of a record.

3. To narrow down your search to only see certain products, you need selection criteria. To see how many products we have that are more expensive than \$100, enter:

```
LIST PRODUCTS WITH PRICE GT "100"
```

GT stands for the Greater Than operator. You can also use **>**

Operators for selection criteria

Operator	Synonyms	Description
=	EQ	Equal to
#	<>, ><, NE	Not equal to
>	GT, AFTER	Greater than
<	LT, BEFORE	Less Than
>=	=>, GE	Greater than or equal to
<=	=<, LE	Less than or equal to
LIKE	MATCHES, MATCHING	Finds a matching word (... is a wildcard)
UNLIKE	NOT.MATCHING	Finds not matching word
SPOKEN	SAID, ~	Phonetic matching

4. Change this view from the default, by adding some display criteria. Enter:

```
LIST PRODUCTS WITH PRICE > "100" CLASS DESC PRICE
```

5. This view is much easier to read. But it's still in random order. Let's change the sort order. Enter:

```
LIST PRODUCTS WITH PRICE > "100" BY PRICE CLASS DESC PRICE
```

Notice that this lists the prices in ascending order (from cheapest to most expensive).

6. To list the highest price first, use BY.DSND (Descending) to invert the order of the records displayed. Enter:

```
LIST PRODUCTS WITH PRICE > "100" BY.DSND PRICE CLASS DESC PRICE
```

You can usually order the Selection and Display criteria wherever you want in the sentence, and MultiValue will figure it out, but this can be open to interpretation. It is best practice to use selection criteria first. First, you describe what you want (the noun: products), then describe more information about the type of items you want (selection criteria: specific products), then what order you want them (sort order: specific order), then what you want to know about these items (display criteria: the fields that are shown about those specific products).

7. We can continue building on this by adding more selection criteria. To see how many products have a weight greater than 25 and less than 30, enter:

```
LIST PRODUCTS WITH WEIGHT > "25" AND WITH WEIGHT < "30" NAME DESC WEIGHT
```

Operands compare two different fields against each other or against any value within a particular field.

8. You can also sort multi-valued fields. Let's start by looking at how they're displayed in the ORDERS file. Enter:

```
LIST ORDERS BY PRODID PRODID DESC
```

Customers can order more than one product on an order. ORDERS, therefore, has multiple multi-valued fields: product ID (PRODID), quantity (QTY), and price (PRICE). On this view, you can see for each Order ID, there are multiple Product IDs. Notice that there is a gap between ORDER IDs indicating there are multiple PRODUCTS.

9. Now enter:

```
LIST ORDERS BY.EXP PRODID PRODID DESC
```

BY.EXP explodes the multi-valued product ID and sorts it that way. Scroll through to page 2 to see raspberries (1001). This is useful to sort multi-valued fields. You can reverse the order, by using BY.EXP.DSND instead.

10. An important part of querying data is searching by dates. Let's look at some:

```
LIST ORDERS BY DATE
```

You can see that the date is formatted as MM-DD-YYYY, and you'll learn more about date formatting in the tutorial 4 on Dictionaries.

11. You can easily look at specific dates with the MM-DD-YYYY formatting. Let's look at orders on April 1st.

```
LIST ORDERS WITH DATE = "04-01"
```

12. You can also use the other operators when looking for dates, such as all orders before or after a specific date. Let's look at the orders we had on and after June 1st.

```
LIST ORDERS BY DATE WITH DATE >= "06-01"
```

13. Sometimes you want to find orders that have no SHIPPED date assigned to them. An empty date is just an empty string.

```
LIST ORDERS BY DATE WITH SHIPPED = ""
```

You can see that when listing orders with no SHIPPED date, that there are still orders having BOTH a shipped date and not a shipped. This is because SHIPPED is a DATE field and a multi-valued field. It shows us which products have shipped and when. Sometimes not all products can be fulfilled when shipping the order.

1.4 Working with the Command Stack

When you use a MultiValue application server, you will most likely be using the Executive Command Language (ECL) sometimes referred to as Terminal Control Language (TCL). You will likely find yourself in a scenario where you want to repeat a previously entered command, change it slightly, or even fix a typo you made. In many modern shells, you can just press the up-arrow on your keyboard and the previously entered command magically appears. In some MultiValue application servers this is not supported. Instead, you can use the command stack.

The command stack is a history of commands you have recently executed. Let's experience how it works.

1. Let's look at a list of products. Enter:

```
LIST PRODUCTS
```

2. To repeat this last query, use the command to execute a command

```
.X
```

3. Enter .L to see the list of recently entered commands.

```
.L
```

This the list of 20 recently executed commands. The most recent command is listed at the bottom and numbered 01.

4. We want to count all product names which are chilies. To search for them, enter:

```
COUNT PRODUCTS WITH NAME LIKE "...Chili..."
```

5. Now we want to search for tomatoes instead of chilies. We can enter the whole command again or use the command stack to speed up the work. Let's use .C to search, match and replace strings. Enter:

```
.C/Chili/Tomato/
```

You'll see the result of the replace in the first line of the command stack.

6. Execute enter this command to execute the modified command.

```
.X
```

You'll now see 5 records counted.

7. The previous command counts the records, but now we want to see what kind of tomatoes we have. We can also use the replace command to change the VERB in the sentence. Enter:

```
.C/COUNT/LIST/
```

then execute your change

```
.X
```

8. Because the query does not define display criteria, we see the default display @ for this file. To only see the NAME field from the PRODUCTS file, we can append the display criteria to the previously executed query. Enter:

```
.A NAME (make sure you do it correctly!)
```

There are two spaces between .A and NAME. The first space separates the .A command from your input. The second space is part of your input because you want to add a space between "...Tomato..." and NAME

Your final query should look like this.

```
LIST PRODUCTS WITH NAME LIKE "...Tomato..." NAME
```

9. Use the command to execute the query with its appendage.

```
.X
```

10. Now list the commands again, and see that there are still only a few entries in the command history. This is because when using .C or .A to change or append a command, you're line-editing that specific command.

```
.L
```

It is also possible to modify a specific line in the entry by specifying a line number.

- **.C2/PRODUCTS/ORDERS/** would change the command on line 02.
- **.A3 NAME DESC** would append a space and NAME DESC to the command on line 03.
- **.X4** would execute the command on line 04.
- **.R5** would recall the command on line 05 and move it to line 01.

There are many more commands you can use to modify the command stack. You can use **.?** to get more information on what you can use on your system, such as deleting sentences, inserting strings, or converting a sentence to uppercase.

Cheat Sheet

You have practiced using LIST, SORT with and without selection & display criteria. It is tough to remember all these verbs, operators and query language. Frankly, you don't need to! You just have to remember where to find all the details of the query language.

We have put together a cheat sheet you can quickly glance at to find the different syntaxes. Additionally there is a larger overview of query commands you can download.

You can download them from here:

[Download the Cheat Sheet Overview of MultiValue query commands](#)

On the next page you will find practice scenarios to exercise querying data. Use these cheat sheets when you need a quick reminder!

2.0 Editing records and fields

In the previous tutorial, you practiced listing the records in different files. In this tutorial you will practice editing records. It is possible to edit those records using the AE Alternative Editor on UniVerse. AE is a line editor and a tool in the terminal to add and edit files, records, dictionaries, and BASIC programs from within your terminal. A line editor only allows you to edit one line at a time, using a command line. There is no full screen and no cutting or pasting. It is one of the basic tools you can comfortably rely on in any U2 (UniVerse/Unidata) application server if you are not using an integrated development environment.

2.1 Using the AE line editor on Universe

In the beginning you made a choice to either use UniVerse Prime, or UniVerse Pick. That choice becomes relevant in this tutorial.

Let's use the line editor

We're going to practice using the AE Alternative Editor built-in the Universe application server.

In this tutorial, you will recreate HAMLET in the line editor to practice creating and manipulating text records in a Universe hashed file.

1. If you do not have the TEST file, create it with the CREATE.FILE command as follows

1. On UniVerse **PICK**:

```
CREATE.FILE TEST 1,2,3 29,1,3
```

2. On UniVerse **PRIME**:

```
CREATE.FILE TEST 3 29 1
```

2. Let's add HAMLET to the TEST file by entering:

```
AE TEST HAMLET
```

This creates a new record in the TEST file with the ID HAMLET.

You are greeted with the AE command prompt:

```
Top of New "HAMLET" in "TEST".
*- - :
```

3. When you are at the editor prompt, press **I** and confirm with **Enter** to enter insert mode. The command prompt changes to indicate the line number you're on.
4. Copy and paste each of the lines below into the editor.

At the end of each line, press **Enter** to go to the next line.

When you are done pasting line 8, press **Enter** to go to line 9.

Press **Enter** on the empty line 9 to return to the prompt mode.

```
001= TO BE, OR NOT TO BE - THAT IS THE QUESTION:
```

002= WHETHER 'TIS NOBLER IN THE MIND TO SUFFER

003= THE SLINGS AND ARROWS OF OUTRAGEIOUS FORTUNE

004= OR TO TAKE ARMS AGAINST A SEA OF TROUBLES

005= AND BY OPPOSING END THEM. TO DIE, TO SLEEP -

006= NO MORE - AND BY A SLEEP TO SAY WE STOP

007= THE HEARTACHE, AND THE THOUSAND NATURAL SHOCKS

008= THE FLESH IS HEIR TO.

5. Use the command `SAVE` to save the **HAMLET** record.

6. Enter **T** to move to the top of the record. The word Top is displayed above the prompt.

7. Enter **P** to print the whole record. The word Bottom is displayed directly above the prompt.

8. Enter **3** to move to the third line. Line 3 will be displayed above the prompt. This is your indication of what line you are on.

9. Enter **+2** to move two lines ahead. You are now on line 5.

When you are entering a digit, you can preface it with a **+** or **-** to move the line ahead or back relative to your current line.

10. Practice moving around the record.

1. Move to the top of the record, and then move to line 6.

are you on the line with *NO MORE - AND BY A SLEEP TO SAY WE STOP*

2. Move back 2 lines.

at line *OR TO TAKE ARMS AGAINST A SEA OF TROUBLES*

11. Move to the bottom of the record with **B**.

12. It is also possible to search in a record with **/**. On the command prompt, enter **/SLEEP** to search for the word SLEEP. This gives you the lines on which this word occurs.

13. You can also search using the LOCATE command **L**. Enter **L/SLEEP** to locate the word **SLEEP** again.

This time you'll move to the line with the first occurrence of SLEEP.

14. Enter **L** to search for **SLEEP** again.

15. When you're editing a record, it is good to see more information on a record

Enter **?** to see all record information.

16. As with the command stack, you can change found occurrences on a line. Move to line 6 and change the word STOP to END with this command:

`C/STOP/END/`

17. Move to the bottom with **B** and enter **I**

18. Add this line to the record:

```
IT IS AN END WE WISH FOR REPEATEDLY.
```

19. Go back to the prompt by giving a return on an empty line.

20. It is also possible to delete a line using the **D** command.

Move to the last line with **9** or **B** and delete this last line.

21. If you want to undo your last action you can use the OOPS command.

Undo the last delete with **OOPS**

Display the record to see the results.

22. You can replace the full line with the R command.

Move to the line you just added and replace it (without using delete) with the following:

```
*--: R/'TIS A CONSUMMATION DEVOUTLY TO BE WISHED.
```

23. You can save the current record with a new name.

```
SAVE HAMLET2
```

24. Now quit the editor with **Q** and confirm you wish to exit without making changes.

25. Perform a LIST.ITEM of the file TEST. Are both records there?

26. Delete the TEST file so it can be created again in a later tutorial

```
DELETE-FILE TEST
```

2.2 Edit data items with AE in Universe

In this tutorial you will edit data items in the MVTUTOR account using AE.

Now that you have practiced adding HAMLET to a TEST file, let's practice how to edit existing records and add new ones with the AE line-editor. We are increasing the price of Lemons.

1. Let's zoom into the PRODUCT information for Lemons. Enter:

```
LIST.ITEM PRODUCTS "7349"
```

2. Field 5, shown on Line 5, shows the PRICE we sell Lemons for. We are increasing the price of Lemons and to do so you can easily edit the record with:

```
AE PRODUCTS "7349"
```

This opens the line editor.

3. Move to line 5 by entering 5 on the prompt.

```
5
```

4. Change 5795 to 5995 with

```
C/5795/5995/
```

or replace the line with

```
R 5995
```

For more explanation on which commands you can use in the editor complete the previous tutorial.

5. File your changes with the **FI** command.
6. List the item to be sure your change was correct.

```
LIST.ITEM PRODUCTS "7349"
```

2.3 Manipulate multi-valued fields

Linda Fleming no longer works for Big-D supermarket. We would like to remove her as a contact.

1. Enter **LIST.ITEM CUSTOMERS "2133"** to list our customer Big-D supermarket in Ropesville.
2. Edit customer **2133**.
3. Move to line **11**.
4. Enter **EV** to open the multi-valued field in it's own editor.
5. Enter **P** to view all four contact names. You can see each multi-valued item is presented on their own line.

We need to delete the third line.

6. Move to line **3**.
7. Delete the item by entering **D**.
8. File the multi-valued field with **FI** and you are returned to the main record.
9. Move to line **12** and enter **EV**
10. Move to line **3**, enter **D** to delete and then file with **FI**
11. Move to line **13** and delete the third value. Remember to file your changes. What was the command to file again?
12. File the whole record.
13. List customer 2133 make sure there are only three contacts: Nathaniel, Michelle, and Gary.

2.4 Copying records

When you want to reuse an existing record you can also copy records. Big-D has opened a new store in Galveston. Let's copy an existing CUSTOMER to create a new one.

COPY can be used to COPY a RECORD from one file to another, from the same file to another key or both.

1. Let's copy the **CUSTOMERS** record **7864** to a new **CUSTOMERS** record **10000**. Enter:

1. On UniVerse **PICK**:

```
COPY CUSTOMERS 7864
```

```
TO: 10000
```

2. On UniVerse **PRIME**:

```
COPY FROM CUSTOMERS TO CUSTOMERS 7864,10000
```

2. You can then edit this entry with:

```
AE CUSTOMERS 10000
```

3. Print the whole record to see that it copied Big-D Tivoli.

4. Move to line **2** and change **Tivoli** to **Galveston**.

5. Replace the address with **2500 Seawall Blvd**.

6. Replace the zip code on line **6** with **77550**

7. Change the credit on line **8** to **0**.

8. File your changes.

9. When you **LIST.ITEM** the record you will see the changes.

3.0 Creating Files

This tutorial will help you understand the structure and types of files and reporting on those files.

This helps you understand the files used in MV and how to add new ones for your application.

In this segment you will:

- See the files that make up a MultiValue account.
- See the folders and files that make up a MultiValue application server and how the VOC makes them MultiValue files.
- Open a file with a text editor to see records.
- Open files within your MultiValue application server.
- Contrast two different file types available in MultiValue application servers.
- Understand the structure of MV database items (Account/File/Record/Fields)

3.1 Display files in the MVTUTOR account

Let's start exploring these files. The first tool, LISTF, shows a list of files available to you from your current account.

1. To list all the Files in the MVTUTOR Account type:

```
LISTF
```

Press any key to continue through the available files.

The LISTF utility lists all the files available to the Account which in this tutorial is MVTUTOR.

Most files will have a DATA and DICT section.

The DATA section obviously contains the data for the file.

The DICT section defines that data for the query language which is used for reporting and selecting of records, it is only needed for queries. The DICT section is usually the same name as the DATA prepended with D_.

LISTF is a listing of the VOC items that refer to the Linux files (binary files) or Linux directories that represent MV Files.

2. Let's see how this looks in the VOC. Enter

```
LIST.ITEM VOC "CUSTOMERS"
```

This shows the item in the VOC for the CUSTOMERS file.

The VOC file in the account contains an item for the CUSTOMERS file, the item in the VOC has three fields:

- F and optionally a description of the file in field 1
- The operating system name for the DATA portion of the file in field 2.
- The operating system name for the DICT in field 3.

The operating system names could be local directory names or full paths.

3.2 Explore Files

Remember the hierarchy for MV: A MultiValue application server has accounts. An account has files. A file has records and each record has fields.

- Account
 - File
 - Record
 - Field
 - Field
 - Multiple value Field
 - Field
 - Multiple value Field
 - Multiple value Field
 - Field
 - ...
 - Record
 - Record
 - ...
 - File
 - File
 - ...

Let's see how this looks on the file system of your application server.

1. You will see the different files and folders that make up this account. These are data files and supporting files. You can recognize the names for the files STATES, CLIENTS and CUSTOMERS among others. Here you see all files that exist in the current directory on the operating

system. The VOC controls which of these files will be exposed in the MultiValue application server.

Use the Linux command `ls` to display the native file system.

```
sh -c "ls -l"
```

2. Now, list the STATES directory, STATES is an operating system directory. In the operating system files you see are the different records. Each one representing a state.

List the STATES directory on Linux using this command from UniVerse

```
sh -c "ls -l STATES"
```

3. You could use Linux tools to update the items since they are standard Linux files in a Linux directory.

```
sh -c"vi STATES/FL"
```

4. Change Florida to FLORIDA if you want

type this exactly while cursor is on F

gUw -- that is lower-case g, capital U and lower-case W

then save the item

:wq -- that is a colon, lower-case w and lower-case q

5. You can now show the STATES records with:

```
LIST.ITEM STATES "FL"
```

See how the name is now all upper case.

6. Lets try to look at a UniVerse hashed file with that same editor.

```
sh -c "vi CUSTOMERS/200"
```

Note the *[Permission Denied]* warning. The CUSTOMERS Linux file is a UniVerse binary file and Linux editors such as vi cannot access the items within it.

Exit the editor without changes

type **:q!** -- That is colon, lower-case q and an exclamation point

MV proprietary files are not directly accessible using operating system utilities such as windows file explorer or notepad. Instead, all proprietary and directory style files can be maintained using MV tools and applications. Proprietary files for MV are stored using a highly efficient hashing algorithm and the contents of record and fields are delimited by an attribute mark.

- an attribute mark is ASCII char(254)
- a line feed is ASCII char(10)
- carriage-return is ASCII char(13)

A simple correlation of MV files to those on the underlying operating system is represented here. Each File in MV is a binary file, or a folder with text files.

[Multi-value structure diagram](#)

3.3 Two principle categories of files

There are two categories of MV files which can be used and maintained in the MV database: Static files and binary files.

All the application files but the STATES file are binary files. The STATES file is a standard operating system level folder, in MV a file, with simple text records containing a one field description.

Directory files (folders) with text files.

The folder name represents the MultiValue file.

Each file within that folder represents a MultiValue record and is a text file that you can edit with any ordinary text editor. Each line is a MultiValue field and the lines are generally delimited by a line feed in Linux or carriage-return line feed in Windows. .

You can use applications on your computer to access these ordinary text files. But directory files are not useful for transaction processing due to access speed.

Hashed files (binary files)

MV has many types of proprietary "hashed" files which provide very efficient high speed access to data, but which can only be altered and reported on via the MV database tools and applications.

The structure of records in the MV environment and types of files

MV hashed files are not directly accessible using operating system utilities such as windows notepad or Linux vi or nano, instead all files proprietary and directory style can be maintained using MV tools and applications. These proprietary files for MV are stored using a highly efficient hashing algorithm and the record fields are delimited by an attribute mark.

The technical reference for a attribute mark is ASCII char(254), Linux files use a line feed ASCII char(10) and Windows uses the line feed and a carriage-return ASCII char(13).

On Windows and Linux, items in a directory file show fields as separate lines when using an editor such as notepad or vi and also in the MV utility AE.

You can use AE to illustrate this. **AE STATES TX** will edit the item, enter **P** to display the item and **EX** to exit.

You cannot use notepad or vi on the CUSTOMERS file. You can with AE, do the same for a proprietary file like CUSTOMERS using **AE CUSTOMERS 4707**.

As stated earlier in this course, AE as well as many other MV utilities can access and work with both categories of files.

3.4 Creating a file on UniVerse PICK

Now that you know how FILES and their DICTIONARIES are represented on a computer, let's experiment creating your own directory type file.

1. To create a new file called TEST enter the command:

```
CREATE.FILE TEST
```

1. On UniVerse **PICK** you are prompted as follows:

- Modulo. Enter **1**
- Separation. Enter **2**
- File type. Enter **3**
DATA file
- Modulo. Enter **1**
- Separation. Enter **1**
- Type. Enter **1**
- File description. Enter **Test file, please ignore**

2. On Universe **PRIME** you are prompted as follows:

- Type. Enter **1**
- File description. Enter **Test file, please ignore**

2. Verify the file is created by viewing the VOC item for the file.

```
LIST.ITEM VOC "TEST"
```

3. Delete the test file with:

```
DELETE.FILE TEST
```

Notice that this verb does not prompt you to acknowledge the delete, so use this verb wisely and sparingly!

4. Recreate the file without prompts:

1. On Universe **PICK**

```
CREATE.FILE TEST 1,2,3 1,1,1 "Test file, please ignore"
```

2. On Universe **PRIME**

```
CREATE-FILE TEST 1 "Test file, please ignore"
```

These instructions create a directory file, indicated with File type = 1. On your computer this looks like a directory in Linux.

5. Display the VOC item again to verify the VOC is the same either way.

```
LIST.ITEM VOC "TEST"
```

Now that you have created the TEST file let's interact with it and add a record.

1. To see 0 records listed enter:

```
LIST TEST
```

2. Add a record to TEST with the sentence:

```
AE TEST FIRSTRECORD
```

This loads the line-editor AE on your application server and shows a prompt. In a later tutorial you will learn about more commands for the AE line editor.

3. At the line-editor (AE) prompt enter **I** to enter line editing mode.

This is indicated with a line number as a prompt.

```
*--: I
001=
```

4. On the first line enter **Test record, please ignore**
5. On the second line press **Enter** to exit line editing mode.

You are returned to the AE prompt

6. Enter **FI** to file the record and exit AE.
7. Now we can use Linux to show the FIRSTRECORD in the Linux Directory TEST

```
sh -c "ls -l TEST"
```

3.5 Create a hashed file on UniVerse PICK

Hashed files are much, much, much quicker for your application server to work with when it comes to creating, updating or searching your data. Let's practice creating a hashed file.

1. We don't need TEST anymore, let's delete it with:

```
DELETE.FILE TEST
```

2. To create a hashed file we just have to specify the correct File Type when creating the file. Most typical database files will work with the file type 3.

Enter the sentence:

```
CREATE.FILE TEST
```

1. On UniVerse **PICK** you are prompted as follows:

- Modulo. Enter **1**
- Separation. Enter **2**
- File type. Enter **3**
DATA file
- Modulo. Enter **1**
- Separation. Enter **2**
- Type. Enter **3**
- File description. Enter **Hashed file, please ignore**

2. On Universe **PRIME** you are prompted as follows:

- File type. Enter **3**
- Modulo. Enter **1**
- Separation. Enter **2**
- File description. Enter **Hashed file, please ignore**

3. Create a record with:

```
AE TEST HASHBROWNS
```

4. Enter line-editing mode with **I**

5. On the first line enter **I like brownies** and give a return to go to line **2**.

6. On the second line give a carriage return to exit editing mode.

7. Enter **FI** to file the HASHBROWNS

8. In the application server directory you can see the hashed file as part of the MVTUTOR account folder.

```
sh -c "ls -l "
```

Notice the TEST does not have a "d" since it is not a Linux directory.

In the tutorial above you've experimented executing the verb CREATE.FILE; but the TEST file isn't something we'd use in our wholesale grocer application. Let's put this skill to the test by adding a file with ZIP codes so that we can better automate order handling in the future.

1. Create a file called ZIPS

1. On Universe **PICK**:

```
CREATE.FILE ZIPS 1,2,3 343,2,3
```

2. On Universe **PRIME**:

```
CREATE.FILE ZIPS 3,343,2
```

2. Run the LOADZIPS program which we have included with the MVTUTOR account. It will populate the file you just created.

```
LOADZIPS
```

4.0 Describing Files, Records, and Fields with a Dictionary

In the last tutorial, you learned how you can edit fields that make up a record or item. Each field is simply a line in the record called field number. But how do you link those field numbers to information that is meaningful? For example, if we want to be able to tell that the first field is a name and another represents an email address we would do so through the dictionary of the file.

The field number indicates the line number for that field in the correlating file.

In this tutorial you will review and maintain the dictionary for a file. A dictionary describes the fields for records stored in a file and is often called a DICT.

To practice, you will create a test file with some sample data. By describing the fields in the DICT as Data-type (D-type) items you will be able to:

- query the data
- apply conversions
- change the field display
- justify data in reports
- identify single and multi-value associations

4.1 Let's display the DICT fields.

1. To see the dictionary item for CUSTOMERS enter the command

```
LIST DICT CUSTOMERS
```

Now, use this to see the fields [Dictionary field map](#)

Now that you know it's a very simple syntax, you'll find it easy to create a DICT item for any given field.

4.2 Describing fields in a DICT

Let's see how the dictionary (DICT) works for querying data by populating the TEST file and adding some sample data to it.

1. Clear the TEST file from the previous tutorial.

```
CLEAR.FILE TEST
```

2. Use the LIST command to see that there are no records

```
LIST TEST
```

3. LIST the DICT of the file and note there is one entry. The @ID is created when the file is created.

```
LIST DICT TEST
```

4. Now add the follow record to **TEST** and file it as **1**

```
AE TEST 1
*--: I
001= Apple
002= Fruit
003= Delicious and healthy!
004= (press <Enter> to exit insert mode)
FI
```

5. Do the same for records **2** and **3**

```
001= Broccoli
002= Vegetable
003= Green and exciting!
```

```
001= Tomato
002= Fruit
003= A fruit but great in a vegetable salad!
```

6. If you **LIST TEST** you will see three results, but only their keys. To see the actual data you would have to specify the fields in your query.
7. Try **LIST DICT TEST** again to see that there is only one field. There are no entries for the three fields in our test file.

To specify the fields representing the items for Apple, Broccoli and Tomato, you can edit the dictionary to specify the fields in the file. You will then be able to query the data.

The Dictionary contains information about the data that is stored in the correlating file. The dictionary for a file is itself also a file. Each record in the dictionary file corresponds to one of the fields or attributes in the correlating file. Each item in the dictionary is syntactically formatted so the query engine can SELECT, SORT and DISPLAY the data in the file. You can use the AE editor to display and maintain records in the dictionary.

8. To add three items to TEST's correlating dictionary, use AE:

```
AE DICT TEST NAME CLASS DESCRIPTION
```

9. For the NAME field in the file add this record to the dictionary. Remember, in AE you can use the back tick ` to create an empty line.

```
001: D
002: 1
003:
004: Name
005: 10L
006: S
```

10. For the CLASS field add this item. Notice how the second attribute is a 2, indicating that this dictionary item refers to the second field number.

```
001: D
002: 2
003:
004: Class
005: 10L
006: S
```

11. For the DESCRIPTION field add this item. The second attribute contains a 3 to refer to the third field number.

```
001: D
002: 3
003:
004: Description
```

```
005: 25L
006: S
```

12. Confirm your dictionary looks like this when you LIST DICT TEST

```
LIST DICT TEST
```

```
DICT TEST      02:55:28pm  29 Jan 2025  Page    1

Field..... Field. Field..... Conversion.. Column..... Output Depth &
Name..... Number Definition... Code..... Heading..... Format Assoc..

@ID           D      0                      Test           10L      S
NAME          D      1                      Name            10L      S
CLASS         D      2                      Class            10L      S
DESCRIPTION   D      3                      Description      25L      S
```

13. To see the data in the test file try:

```
LIST TEST NAME CLASS DESCRIPTION
```

14. When you try LIST TEST you still only see the normal keys. We can use the special @ field in the dictionary to indicate which fields we want to display by default. To add this dictionary item use:

```
AE DICT TEST @
```

15. The @ is the attribute that defines the default report fields, and is a PHrase. Phrases are a dictionary type just like the Data-type (D-type) fields that you created for Apples, Broccoli, and Tomatoes.

```
001= PH
002= NAME CLASS DESCRIPTION
```

16. We can see that the Description is wrapping because we only specified a 25-character length. Let's give the description a bit more space by editing its dictionary item with:

```
AE DICT TEST DESCRIPTION
```

17. On the fifth line for Field format change **25L** to **50L** and file the dictionary item.

18. Now try LIST TEST again and see that the description now has more space to be displayed.

You have added three items to the dictionary file for TEST. Each of these items describe the fields for each item in the file TEST.

Each attribute of the DICT item contains specific data. This table explains in detail what each of the seven fields of a dictionary item represents.

Field Number	Description	Valid Entry
001	DICT Type	D - Data field or I - Type Calculation or PH - Phrase

002	Attribute or Calculation or Phrase	
003	Data Conversion	Conversion to apply to output data, like D2/ for date with a 2 digit year and slash delimiters
004	Field Heading	Heading to appear on reports
005	Field format	Length and justification
006	Multi-valued or single value field	M - MV field or S - Single value field
007	Association	MV PHrase field if this DICT relates to other MV fields

4.3 Editing the ZIPS dictionary

In a previous exercise you have created a file called ZIPS and imported zip codes into it. Now lets use the DICT of that ZIPS file to learn some more about DICT items. You will look at the existing DICT item definitions of the ZIPS file and improve them.

1. Compare the output of these two commands: `LIST.ITEM ZIPS`

```
LIST DICT ZIPS
```

Notice that each line listed by LIST.ITEM is represented by @ID, F1 through F15 and GPS when viewing the results of LIST DICT

2. You can see each item in the dictionary of the ZIPS file with this sentence. Enter:

```
LIST.ITEM DICT ZIPS
```

3. Notice in **LIST DICT ZIPS** that each of the field names are named **F1** through to **F15**, with the exception of **GPS**. This makes it harder to list, select or display the data because you can only use this field name as a key in your queries.

You would have to know by heart that F2 refers to the Primary City

4. To see a listing of all zip codes with the Type, State and GPS columns you would like to be able to do:

```
LIST ZIPS TYPE STATE GPS
```

When you try this right now, it does not work because there are no dictionary items named TYPE or STATE that the query engine can understand.

5. Instead, try:

```
LIST ZIPS F1 F5 GPS
```

This gives you a good result; but is that really usable? It is the same problem as our Apples, Broccoli, and Tomatoes in the previous lesson; but this time there are fields in the DICT. You can fix this by renaming the items in the dictionary to better represent the data.

This is a common problem you might encounter where fields are not described properly in databases. You can fix that by renaming existing fields or adding synonyms of them. In the next steps you'll

practice both.

6. Let's use the AE editor:

```
AE DICT ZIPS
```

You are now prompted for a Record ID.

Type **F1** to edit the dictionary item defining zip code type.

Enter **SAVE TYPE** to save this record with the key TYPE.

You have now made a duplicate of the F1 record. Exit the **F1** item.

7. Check **LIST DICT ZIPS** or **LIST.ITEM DICT ZIPS** to see that for Field Number 1 there is **F1** and another dictionary item called **TYPE**.

8. You can now use **LIST ZIPS TYPE** to query the Type field semantically.

By saving an existing dictionary item with another name you create a synonym. This is useful in databases that have their fields listed with F and a number.

Another way to create a synonym is to use COPY to copy an existing dictionary item without the use of AE

9. To copy **F2** to the key **PRIMARY_CITY** enter:

1. On Universe **PICK:**

```
COPY DICT ZIPS F2  
TO: PRIMARY_CITY
```

2. On Universe **PRIME:**

```
COPY FROM DICT ZIPS TO DICT ZIPS F2,PRIMARY_CITY
```

You can now use LIST ZIPS PRIMARY_CITY in the query language.

If you want to change more information in a dictionary item you can edit all the DICT items by using asterisks (*).

10. Enter **AE DICT ZIPS *** to edit all the records in the dictionary.

This will take you through all the items in the DICT ZIPS. Observe line 004 and add DICT items using SAVE to save a copy then Q to quit or simply Q to quit if you do not want to SAVE under a new name.

For example, you could SAVE ACC_CITIES for the item with field four of ACCEPTABLE_CITIES.

Repeat this step for each of the F## fields, reviewing the column heading on line 4 and using that to save a new record. This leads to a dictionary that has different field names for the different field numbers.

A dictionary is allowed to have multiple items per field number because it is the number in the dictionary item that indicates the position of a field in the correlating file.

12. If you do not want synonyms for a field and want to keep a clean dictionary, you can use CNAME to rename items in a file. Since the dictionary is a file, we can use CNAME to rename the items that describe our fields.

```
CNAME DICT ZIPS F4 TO UNACC_CITIES
```

This would remove the F4 definition and instead only leave the UNACC_CITIES for this field. That might be problematic in production systems that might use the original F4 definition in program logic.

4.4 Converting computer data to help humans understand it

There are some amazing things we can do with the DICTIONARY. Let's look at Conversions first.

Data in a file might not always be stored in a human readable format. Look at dates, we count dates since the end of 1967 to figure out what day it is today. Would you know what date the day 11689 is? Of course not! You need to convert this computer data to find out that it's January 1st 2000. Other conversions may simply extract portions of data or change display of text such as casing.

You can do this with Field 003 in the dictionary to specify what kind of conversion you want. In the next segments you will practice several data conversions.

4.5 Date conversions

Date conversions allow the internal stored data for multi-value databases to be returned in a human readable format.

In this video Dan explains the most commonly used DATE conversions and shows how to apply them.

[How can I use Date Conversions](#)

The number after the D signifies the number of digits to display for year and the dash is the separator for the month, day and year.

The **D2-** would return 12-31-1967 for 0, 01-01-1968 for 1 and 12-30-1967 for -1. The raw data number 20090 would return as 01/01/2023.

Let's create your own date conversion. The ORDERS file currently lists the date numerically with dashes.

1. To see the current conversion for the field DATE do:

```
LIST-ITEM DICT ORDERS "DATE"
```

You can see that it has D4- as the conversion on line 003. This will display the date numerically with 4 digits for the year. It is separated by the dashes

2. To see the raw computer data on Field 1 do this

```
LIST-ITEM ORDERS "1010"
```

3. To see the converted raw data as a date enter

```
LIST ORDERS DATE '1010'
```

The value in line 1 which is the Date displays as March 11th

It shows that month and day with a dash and 4 digit year. This is how D4- converts the raw computer data. A date with 4 digit year separated by dashes (-).

4. Sometimes you want to see the month listed instead of the numerical values. Let's create a DICT item called MONTH to show us the month. Enter

1. On Universe **PICK**

```
COPY DICT ORDERS DATE  
TO: MONTH
```

2. On UniVerse **PRIME**

```
COPY FROM DICT ORDERS DATE,MONTH
```

5. Edit the item with

```
AE DICT ORDERS MONTH
```

Change Field 003 from **D4-** to **DMA**

Change Field 004 from **Order²Date** to **Month**

File the item

6. You can see that the name of the month has been extracted if you now

```
LIST ORDERS MONTH
```

Here are the most generally used codes for date conversions

Code	Description
D	Returns generic 01 JAN 2000 format
DD, DM, DY	Return the individual Day, Month or Year
DWA, DMA	Returns the date-of-week or month as a word

4.6 Time conversions

MT, MTS, MTH or MTHS Time conversions are common for many applications and the MT conversion is used to return the time from the raw number of seconds.

MT alone will return that number of seconds as hours and minutes.

MTS will return it with hours minutes and seconds.

Time is often in the realm of a day and that is where the H will return the number as AM/PM rather than the hours, minutes in seconds often referred to as military time. The raw data number 50000 would return 13:53 if MT was used, 01:53PM if MTH, 01:53:20PM for MTHS and 13:53:20 for MTS

You can display time as hours, minutes and/or seconds, and in 24-hour or 12-hour notation with and without AM or PM suffix.

1. To see a list of detailed ORDER information enter the command

```
LIST ORDERS DETAILS TIME
```

2. You can see the Order date and Order time. Let's look at order 1010 again,

```
LIST ORDERS DETAILS TIME '1010'
```

3. If you look at the raw item 1010 with

```
LIST.ITEM ORDERS '1010'
```

You will see the amount of seconds that have elapsed since midnight on the day that this order was entered.

4. By default we're displaying the time in a 12 hour notation. So five o' clock at night is displayed as 5:00 PM. Sometimes you want to display the time in 24-hours notation. So that'd become 17:00. Let's edit the dictionary item for TIME with

```
AE DICT ORDERS TIME
```

5. On Field 003 you can see that the conversion code is MTH which will list the time in a 12 hour notation with an AM or PM suffix.

Change Field 003 to **MT** to show the data in a 24-hour notation.

6. To see the TIME in 24-hour notation check with

```
LIST ORDERS DETAILS TIME '1010'
```

4.7 Masking decimal conversions

Usually numbers are stored as integers. The masked decimal conversion codes convert a number to an external format. Allowing you to scale, insert a currency symbol, or define thousands separators.

MR2,\$ or MD2,\$

This conversion would return \$1,234.56 if the raw data was 123456. This is a mask code for numbers and will start with M usually followed by R for right or D for decimal which is synonymous to R for right.

Let's see this in action in your application server by reusing the TEST file we created in the previous tutorial. There you created three records, saved as 1, 2, and 3.

1. To see the three records: Apple, Broccoli, and Tomato do a

```
LIST TEST
```

2. Now let's add a fourth field to each of the records, one resembling the numeric value for carbohydrates in grams.

```
AE TEST 1 2 3
```

This will open AE with multiple records.

3. On Field 4 for Apple add the number **1381**

Remember, you can enter FI to file the record and move to the next one.

4. On Field 4 for Broccoli add the number **664** and file your changes.
5. On Field 4 for Tomato add the number **39** and file to exit.
6. If you **LIST TEST** you won't see the data you've entered, because it's not in the dictionary yet.

To see the details of the Apple record do

```
LIST-ITEM TEST "1"
```

7. Now let's add this fourth field to the attributes in the dictionary. Enter

```
AE DICT TEST CARBS
```

Describe the field as follows:

```
001= D
002= 4
003= MD0
004= Carbohydrates (g)
005= 5R
006= S
```

8. Do

```
LIST TEST NAME CARBS
```

You will see the raw numbers, because MD0 applies no masking to the stored number.

But it does not make sense for an apple to have 1381 grams of carbohydrates when itself is barely a few hundred grams heavy. We need to apply a masking code to show the number correctly.

9. Edit field 003 to say **MD2,\$** with

```
AE DICT TEST CARBS
```

Make sure that you type a comma (,) between MD2 and the currency (\$) symbol.

10. When you **LIST TEST CARBS**, you'll see the number represented as money. That's the work of the dollar sign. The dollar sign in the masking code represents currency symbol and means that the configured currency symbol will be prepended to the result. The default is a dollar sign, but you can change the currency of your application server.
11. Apply masking code **MD2**, and that will eliminate the dollar sign.
12. A valid example of MD2,\$ is the price of products.

```
LIST PRODUCTS PRICE
```

13. The third position in the masking code, the 2, represents the number of digits to the right of the decimal. We want to display 3 decimals for the COST field. Tenths of a penny are important when we pay our vendors for large quantities.

```
LIST PRODUCTS PRICE COST
```

This shows you the differences between MD2,\$ and MR3,\$ where both of their raw data is stored as normal integers. With the digit you can determine how large or small the stored integer is.

The comma is optional and specifies that a thousands separator is to be put in. The default is a comma, but you can configure your application server to use periods instead.

There are a myriad of options explained on the knowledge base, but in general MR2,\$ is how the MD and MR mask conversion is used.

4.8 Extractions of partial data in a field

There are two common conversions that extract a portion of a data field.

Text extraction

1. A simple extraction is text extraction by character position T2,3 would Text extract starting at character 2 for 3 characters.

Lets extract those characters from the ZIP in the CUSTOMERS file

```
LIST CUSTOMERS ZIP ZIP CONV "T2,3"
```

Group extraction

1. The raw data in customer field 11 has contacts lastname, firstname. `LIST-ITEM CUSTOMERS`
2. `CONTACT_LNAME` uses a group extraction to show only the list name. Display that DICT item.

```
LIST.ITEM DICT CUSTOMERS "CONTACT_LNAME"
```

This DICT uses the group extract conversion G0,1 to extract the last name

The group extract conversion is best understood as follows:

G0,1 -- extract the group by skipping **0** elements delimited by comma (,) and take **1** element.

4.9 Differences between conversions and raw data

In this video Dan explains the difference between a conversion and raw data. This video uses the PRIME syntax for COPY and you may need to use the PICK syntax as shown in earlier tutorials.

[What are the differences between a conversion and raw data](#)

4.10 Masking characters or changing presentation of text

There are common character conversions that change the case of alphabetic fields.

There is MCU for UPPER, MCL for lower and MCT for Mixed case.

1. See this using the CLASS field in PRODUCTS. Perform the following queries.

```
LIST.ITEM PRODUCTS
```

Field 003 shows what the DICT CLASS defines

2. Show that field in uppercase

```
LIST PRODUCTS CLASS CONV "MCU"
```

3. Then in lower case

```
LIST PRODUCTS CLASS CONV "MCL"
```

4. And then in mixed case

```
LIST PRODUCTS CLASS CONV "MCT"
```

These are all handy conversions to change the display of a field. Notice that you did these conversions inline. You can use the CONV keyword to do a conversion on the command line without needing to add or change a dictionary item!

4.11 Field length and justification Field 5 of the DICT item

The numeric portion of field 005 specifies the wrap length for a field, data returned is simply wrapped to the next line after this length. The field justification and length determines both the look of the returned data as well as the sort order.

Generally numeric fields are sorted right-justified and alphabetic fields are not. This affects how a field is sorted, in that numbers 1 through 9 would sort prior to 10 if right-justified, but numbers, if left-justified, would sort with those starting with 1 first, then 2 then 3 etc, the numbers 1,2,3,10,100,110 would sort as 1, 10,100,110,3 if they did not have a field justification of R for right.

We have a problem in our dictionary. When we try to sort our customers by their unique number the results are odd. Our house account shows up somewhere in between customers 1000 and 2000. Why is that?

1. See the customers weird order with

```
SORT CUSTOMERS NAME
```

You'll see MVTUTOR Grocery Wholesalers before the Family Grocers.

2. Look at the dictionary listing for CUSTOMERS and the @ID field.

```
LIST DICT CUSTOMERS
```

You can see that the Field length and justification is set to 10L. This is used by your application server to determine the sort order. Numeric fields that are left-justified will sort like strings of text. Meaning 200 will be sorted with the 2000's.

3. Change the DICT for the @ID field to say 10R with

```
AE DICT CUSTOMERS @ID
```

4. Now sort the customer file

```
SORT CUSTOMERS NAME
```

The query should now return list sorted correctly by numbers. This is because your MultiValue application server takes justification in account when sorting numbers.

Make sure that you know that the length and justification of a field determines the sort order of your queries. With that in mind, generally if data which is a number is to be sorted, as we generally assume numbers to be sorted, they would need to be R-justified.

E.g.: Dates should be right-justified because they are a number. This is often overlooked by most developers and a reason for many support tickets.

This becomes especially important when you start writing BASIC programs to work with SELECTLISTS. More on those later.

Data can also be:

- L for left justification
- C for center justification
- T for text justification

The L and C are obvious, the T is simply the same as L with logic to wrap words so if possible words are wrapped as a whole like a word processor.

5.0 Creating Interactive (I-type) fields

So far you have been working with Data (D-type) fields. You created entries in the dictionary so that you can query semantically, you have formatted them with headings and justification, and converted the machine data to a human readable format. Now you will learn how to unlock more power from the application server by using Interactive (I-type) fields.

This tutorial will help you understand Interactive (I-type) fields in the dictionary that allow for more than just simple field display and formatting like the Data (D-type) fields.

This helps you understand when a more extensive DICT is needed and then how to create that DICT.

In this segment you will:

- Explore existing I-types in the DICT files included with this tutorial
- Understanding the code on field 2 of the DICT
- Some simple examples
- Using BASIC with the SUBR function

5.1 Understanding the code:

Let's create your first I-type field. Field 002 is the heart of the Interactive field, sometimes called I-Descriptor, and it differs from the D-type. It includes code rather than just a field number and you can see it as a formula that operates on the data for you.

In our ORDERS file we want to know the extended price for the amount of products ordered.

1. Create an I-type field with

```
AE DICT ORDERS EXT
```

2. File the following definition. We know that the extended price is simply the quantity multiplied by the product price.

```
001: I
002: QTY * PRICE
003: MD2,$
004: Extended Price
```

```
005: 10R
006: M
```

3. See the extended price with

```
LIST ORDERS DESC QTY PRICE EXT
```

There are more calculations we can do, we want to see how old the orders are.

1. Create an I-Type field AGE with the following definition

```
001: I
002: @DATE - DATE
003: MD0,
004: Age in days
005: 5R
006: S
```

2. List the data with **LIST ORDERS DATE AGE**

There's more we can do. Sometimes we want to concatenate strings together. This is easy to do and something we do in our MVTUTOR account.

1. Look at the CLIENTS raw data with **LIST-ITEM CLIENTS "94616"**

You can see that we store Tim's first and last name in the first two attributes. Sometimes we want to use two values in a single field, to make long queries or basic programs easier to read and understand. This is why we have an I-type field NAME that concatenates the two D-type fields.

2. Do **LIST CLIENTS "94616" FIRST_NAME LAST_NAME NAME**

You will see the three values separately.

3. Inspect the I-type with

```
AE DICT CLIENTS NAME
```

Strings are concatenated with a colon. You can enter literal strings in between quotes.

Often times you will encounter that names and addresses are concatenated. There might be other strings that you want to concatenate into a new string when dealing with your data.

On UniVerse you might see more than 6 lines. This is because by default UniVerse will compile the I-type field into the DICT item itself. Lines 14 up to 20 are the compiled code and show up in your terminal.

4. Edit Field 002 to say:

```
FIRST_NAME; LAST_NAME; @1 : " " : @2
```

This also concatenates the two fields and instead uses them as an @variable. @1 and @2 simply refer to groups 1 and 2 that have been defined in a code with a semi-colon. This allows you to create more extensive formulas and reuse results. You'll look at that a bit later in this tutorial.

It is possible to use all kinds of information sources and operators in the code of an I-type field, to create extensive formulas.

You can use:

Information Sources	Operators
D-type fields in the current file	Arithmetic
Other I-Type fields in the file	String manipulation
System variables	Relational comparisons
Constants and literals	Logical operations
Basic functions	Conditional operations

5.2 Exploring I-Type DICT items in the MVTUTOR account

Let's start exploring the I-Type fields that are available in this MVTUTOR account.

1. Type **LISTF** to remember which files are available to you from your current account. From that list we can start filtering those files that have I-type fields.

Usually you will start work on an existing database and with this method you will be able to explore the account and figure out the Interactive fields.

2. You can create a list of files that have I-Type fields in the DICT by searching through the DICT. Because the DICT is also just a normal file, you can use query parameters. Enter:

```
LIST DICT CLIENTS WITH F1 LIKE "I..."
```

The verb LIST lists the matched records completely. The DICT keyword searches the items of the dictionary. CLIENTS is the file to search items in. The WITH clause helps us filter the items.

The F1 field is the key for Field 001 in the dictionary. Remember that this one is a D when it is a Data Type field. In this case we want to find I-type fields. So we start by looking for fields that start with I.

3. The magic is in this query:

```
LIST DICT filename WITH F1 LIKE "I..."
```

You could do this in any MV account and discover other I-type DICT items. If you continue this query for all the files you can find of which the following files have I-type fields.

- CLIENTS
- CUSTOMERS
- ORDERS
- PRODUCTS
- RECEIVABLES

4. You can see that Field 002 in the dictionary contains code. When a query will display an interactive field, that code is executed on the application server. This becomes immediately apparent in the PRODUCTS file where we can calculate the total value of that product in our inventory using an interactive field.

```
LIST PRODUCTS NAME PRICE QOH VALUE
```

A good way to explore I-types and create your own is to display an existing one and then copy that to a new item and make the changes that pertain to what you want for your new DICT item.

5.3 Extensive formulas

Now that you know about some basic functions, let's look at those more extensive formulas now. Oftentimes you want to perform several operations on the data in a record, especially if they are multi-valued. Using the EVAL keyword we can build out the different parts of a formula and then combine them in an I-type field.

1. We're going to add a sales tax, an excise tax and a \$500 transportation insurance surcharge for orders over \$5,000. For this we'll need to create the TOTAL_ORDER field.

We can use the formula `ITEMTOTAL + (ITEMTOTAL * .035) + (ITEMTOTAL * .015) + (IF ITEMTOTAL > 500000 THEN 500 ELSE 0)`

Test this formula with:

```
LIST ORDERS ITEMTOTAL EVAL "ITEMTOTAL + (ITEMTOTAL * .035) + (ITEMTOTAL * .015) + (IF ITEMTOTAL > 500000 THEN 500 ELSE 0)"
```

2. A forgotten parenthesis would give you a totally different result. Let's break the formula into multiple steps, beginning with the sales tax. To calculate the sales tax you multiply the ITEMTOTAL by .035. See this with:

```
LIST ORDERS ITEMTOTAL EVAL "ITEMTOTAL * .035"
```

3. The second part is to calculate the excise tax, which is multiplying the ITEMTOTAL by .01

```
LIST ORDERS ITEMTOTAL EVAL "ITEMTOTAL * .015"
```

4. The last part is a bit more complicated. We want some conditional logic when the ITEMTOTAL is more than \$5,000.00

```
LIST ORDERS ITEMTOTAL EVAL "IF ITEMTOTAL > 500000 THEN 500 ELSE 0"
```

5. Now that you have seen the three parts at play let's combine them to calculate the real TOTAL_ORDER field. Create a field TOTAL_ORDER with

```
AE DICT ORDERS TOTAL_ORDER
```

6. First complete lines 1, 3, 4, 5 and 6 as follows. We'll do the code in the next step.

```
001: I
002:
003: MD2,$
004: Order with Tax
005: 10R
006: S
```

7. On Line 2 first calculate the sales tax:

```
002: ITEMTOTAL * .035
```

8. Add a semi-colon to end the first group:

```
002: ITEMTOTAL * .035 ;
```

9. Add the formula to calculate the excise tax behind the semi-colon, and include another semi-colon to end the second group.

```
002: ITEMTOTAL * .035; ITEMTOTAL * .015;
```

10. Calculate the surcharge as the third group:

```
002: ITEMTOTAL * .035; ITEMTOTAL * .015; IF ITEMTOTAL > 500000 THEN 500 ELSE 0;
```

11. Now finish the calculation by adding the three groups to the ITEMTOTAL

```
002: ITEMTOTAL * .035; ITEMTOTAL * .015; IF ITEMTOTAL > 500000 THEN 500 ELSE 0; @1 +  
@2 + @3 + ITEMTOTAL
```

12. Try the new field with:

```
LIST ORDERS ITEMTOTAL TOTAL_ORDER
```

The sales total for order 1212 should be \$3,817.36.

For order 1316 (which is over \$5,000) it should be \$6,105.53.

You can see that by grouping your operations together, you can make your formulas much more readable. We do recommend that when your formulas become so complex you need many groups: try thinking about creating a BASIC subroutine and calling that instead. You'll see that covered later.

5.4 Cheat sheet for Interactive fields

[Download cheat sheet on I-type fields](#)

5.5 Displaying data from other files with the TRANS keyword

Often you want to display data from multiple files at the same time. You can use the TRANS keyword for this.

1. When we LIST a customer we can see their STATE, and we would like to see the full name of that state.

```
LIST CUSTOMERS "7777"
```

2. We would like to see their full state name, which we store in the STATES file.

```
LIST STATES
```

3. Look at the fields of the STATES file with

```
LIST DICT STATES
```

Notice that NAME is Field Number 1.

4. We can display data from the STATES file when listing data from CUSTOMERS. Using the TRANS keyword we can translate the STATES file with the STATE field in CUSTOMERS. Take a look at the STATE_NAME field in the CUSTOMERS file:

```
LIST DICT CUSTOMERS "STATE_NAME"
```

```
002 TRANS(STATES,STATE,1,'X')
```

look at the STATES file, which is the first argument of the keyword.

The second argument links the STATE field of CUSTOMERS to the STATE_NAME field.

The third argument tells TRANS the field number to retrieve back.

The fourth argument tells TRANS with an 'X' to not return anything if the data is not found.

5. To see the customers and their full state name enter:

```
LIST CUSTOMERS STATE_NAME
```

6. It is very easy to create queries that include data from multiple files. Let's display a customers phone number when we list an order. Let's first inspect the CUSTOMERS file to figure out which field we need.

```
LIST DICT CUSTOMERS "STOREPHONE"
```

You can see that the Store Phone field is Field Number 10.

7. We have the CUSTID in the ORDERS file. Let's start building the I-type field STOREPHONE in ORDERS.

```
AE DICT ORDERS STOREPHONE
001: I
002: TRANS(CUSTOMERS, CUSTID, 10, 'X')
003:
004: Store Phone
005: 14R### ###-####
006: S
```

8. File the new field as the DICT item STOREPHONE

9. Test the new field with:

```
LIST ORDERS DETAILS STOREPHONE
```

Now we should easily be able to call our customers when we are processing their ORDER.

5.6 Understand command line options that allow for alteration of data display during query command

In the previous tutorials you have inspected, created, and altered entries in the DICT. It isn't always necessary to use the DICT to alter the output of a query, sometimes you can do this from the command line itself. This lesson will help you understand the tools available at the command line to alter the output of a query without needing to alter the DICT items.

Look at this table of keywords:

Keywords

Keyword	Description	Changes output of	Notes
---------	-------------	-------------------	-------

HEADING	Report heading which defaults to command with page/date	Overall Report	Must be in quotes and special characters allow for embedding date/time/page and data
FOOTING	Default is no footing except if to screen, then "Press any key to continue..."	Overall Report	Same rules as HEADING
COL.SPACES	Change spacing between columns on the entire report	Overall Report	Number cannot be in quotes and range starts at 0
EVAL	Create a phantom DICT item as though this was the I-type code	Field 002	Must be in quotes. If quotes already used, then use apostrophes or back-slashes
CONV	Change conversion used	Field 003	Must be in quotes. Can use "" to nullify conversion already in DICT item.
FMT	Change justification and length	Field 005	Quotes are optional
SINGLE.VALUE	Treat field as S	Field 006	
MULTI.VALUE	Treat field as M	Field 006	
ASSOC	Change/Add Association for multi-values	Field 007	Must be in quotes

The table above shows that the attributes of a DICT item can be altered in your query.

The exception is EVAL, which does not replace Field 002 of an existing DICT item, but instead is treated AS the dictionary. The other keywords alter the properties of a stated DICT or EVAL.

Exploring the keywords with examples

Let's see this table in action.

1. Start with this statement

```
SORT CUSTOMERS NAME STORE CITY STATE ZIP CREDIT
```

After each change, see the new output by using **.X** to execute the changed statement

2. Now change the heading by adding the following to the end of your query

```
.C/CREDIT/CREDIT HEADING "Alphabetic list of customers and stores"
```

3. Add a FOOTING

```
.C/stores"/stores" FOOTING "This report is not to be distributed outside of the organization"
```

4. And the last of the overall report modifiers, we can change the column spacing to 2

```
.C/ion"/ion" COL.SPACES 2
```

5. The most common change would be the column heading. Change CREDIT column heading to "Buying power". Often fields are known by different names depending on department, for example CREDIT limit to accounts payable is a limit whereas in a sales report for salesman it may mean buying power.

```
.C/CREDIT/CREDIT COL.HDG "Buying power"
```

6. Change the STORE column to wrap at 15 charactersThe next most common thing would be the format which is the length and justification. Sometimes the columns on a report are too wide for the medium where they are displayed. You do not want to change the actual DICT size which will affect all reports using the DICT field, but you may need it shorter or longer for a specific report.

```
.C/STORE/STORE FMT "15L"
```

7. Let's change that to word wrap rather than just split the line at 15 characters

.C/15L/15T

8. Show the credit limit as thousands without pennies. The conversion or CONV of a field relates to how MultiValue databases store fields internally and how they are to be displayed or converted for display. These are usually date, time, numeric or text conversions. The CREDIT field on this report has a MR2,\$ conversion because the amount is stored as a integer number of cents and when displayed we want it to display with 2 digits after the decimal and a comma between 1000 place holders and a dollar sign on the front.

```
.C/CREDIT/CREDIT CONV "MR02,$"
```

9. Add an underline field using EVAL as a new column. You can add a report column using EVAL and in this case, we simply want to put a underline so we can check off names and have a heading of Check?

```
.C/SPACES 2/SPACES 2 EVAL "\_____\" COL.HDG "Check? "
```

Note that back slashes (/) are inside the quotes with the 10 underline characters.'

or use the STR function from lesson 2

```
.C/SPACES 2/SPACES 2 EVAL "STR(' ',10)" COL.HDG "Check?"
```

Here is the final statement with all the changes

**SORT CUSTOMERS NAME STORE FMT "15T" CITY STATE ZIP CREDIT CONV "MR02,\$"
COL.HDG "Buying power" HEADING "Alphabetic list of customers and stores" FOOTING
"This report is not to be distributed outside of the organization" COL.SPACES 2 EVAL
"STR(' ',10)" COL.HDG "Check?"**

Now compare that to what you started with

Sort Customers Name Store City State Zip Credit

The nice thing is that, in a legacy system with 1000's of dict items, you can change from the command line what you want, instead of creating a whole new DICT. You need only create new DICTS for things that are going to be repeated.

Exercise

Complete this exercise towards your certificate.

1. Click the link to open the exercise in a new window.

[Creating Interactive \(I-type\) Fields](#)

2. Switch between the exercise in another window, and this window to complete the exercise.

6.0 Manipulating Selections of Data with SELECT LISTS

When you perform a query you can create lists with them for reuse. These lists are usually the primary keys (the record IDs) to be used by subsequent processes. The lists are kept in a system buffer and that buffer can be saved in a designated directory and recalled or edited by name. The list can be immediately used by an application process, feed a report query, or be saved for later repeated processes. We call this a SELECT list.

6.1 Creating the list buffer and save/get that list

SELECT and SSELECT are similar to the LIST and SORT processes, but instead of producing an output report they store that output in a buffer which can:

- be processed by a subsequent command,
- saved with the SAVE.LIST command
- ignored and reset with CLEARSELECT.

Let's see how SELECT lists work in action.

1. Type the following to produce a display of the primary keys of the CUSTOMERS file.

```
LIST ONLY CUSTOMERS WITH NAME LIKE "B..."
```

Note that the ONLY is used to assure that only the primary key is listed and that the @ phrase (default listing) is not processed

You will see 11 customer primary keys listed.

2. Now, type this to create a list of the primary keys from the CUSTOMERS file and hold them in a system buffer to be used by the next process or command.

```
SELECT CUSTOMERS WITH NAME LIKE "B..."
```

Notice the 11 records are "selected", but that is the only display. You have created your first list. The list is somewhat temporary in that it is being held in a system buffer awaiting the next command. Your MultiValue application server indicates this with an altered prompt. In UniVerse you can see you're working with a SELECT list by two prompt characters:

```
>SELECT CUSTOMERS WITH NAME LIKE "B..."
11 record(s) selected to SELECT list #0.
>>
```

3. Lets try these commands:

```
LIST ONLY CUSTOMERS
```

What displayed? Does it look familiar? Now that you have executed a command that uses the SELECT list the prompt changes back to a normal one.

4. Type it again or perform **.X**

```
LIST ONLY CUSTOMERS
```

This query now shows all the customer primary keys. The temporary buffer is no longer there and your application server performed the query against the file instead of the selected list.

5. Let's do the SELECT again and then CLEARSELECT. This time, notice how the command prompt changes again.

```
SELECT CUSTOMERS WITH NAME LIKE "B..."
```

```
CLEARSELECT
```

The select temporary buffer was cleared, now the command prompt goes back to "normal". Just like when you executed a query on the selected list.

6. The next command makes sense because you probably want to save that temporary buffer so it can be recalled without having to process all the records again. To use a saved list repeatedly you need to save it to a file. Let's recreate the list in the buffer and then save it.

```
SELECT CUSTOMERS WITH NAME LIKE "B..."
```

```
SAVE.LIST B_CUSTOMERS
```

Be aware that when you SAVE a list-name, in this case B_CUSTOMERS, must be unique or it will overwrite the list already existing with that list-name. The designated directory for saved lists is shared by everyone.

7. Now you have GET.LIST available to retrieve that list of records without having to re-scan and select the original file. Re-establish the temporary buffer with:

```
GET.LIST B_CUSTOMERS
```

The temporary buffer now has the same primary keys that the original SELECT CUSTOMERS WITH NAME LIKE "B..." produced. The selected message comes up and the command prompt changes, just like the original SELECT.

Working with saved lists

There are three commands to edit, delete and copy saved lists:

Command	Description
EDIT.LIST	The EDIT.LIST can be used to alter an existing list created by SAVE.LIST or can create a new list from scratch. EDIT.LIST list-name does not have to be an existing list, if list-name does not exist, it will be created when the item is filed. The EDIT.LIST uses the simple MV line editor (ED). When the item is filed, it will be a standard list available for all the list commands.
DELETE.LIST	DELETE.LIST list-name will delete a stored list

COPY.LIST	COPY.LIST will prompt for TO: at which point you can copy it to another list name or copy it to a file and item name by using the syntax "(filename item-name". Note that the left parenthesis is mandatory.
-----------	--

6.2 Why use a saved list?

To allow applications to process a specific list of items in a specific order, use a saved list.

Lists are needed by some applications to retrieve a list of primary keys to process. For example, a list could be used to determine ORDERS to print for the day, or to create a list of CUSTOMERS in a specific city to receive a marketing email.

The process may simply `SELECT ORDERS WITH DATE = "11/02"` and process them. It could also run the `SELECT` once and `SAVE.LIST DAILY.ORDERS`. This would allow subsequent processes to `GET.LIST DAILY.ORDERS` to print the orders and then, if they print correctly, reuse that list with `GET.LIST DAILY.ORDERS` to update those orders.

This logic is used in many applications. In this simple BASIC program, we read the items from the list and print them to the terminal. Although this actual code has little use, it represents the logic that many applications written in BASIC will use to process a list of selected items.

```
AE BP DISPLAY_ITEMS
001 LOOP
002 READNEXT LISTLINE ELSE EXIT
003 CRT LISTLINE
004 REPEAT
```

Review of the commands that process lists

Command	Description
SELECT / SSELECT	Command to create lists of primary keys and save them to a system buffer
SAVE.LIST list-name	Save items from an earlier SELECT as list-name
GET.LIST list-name	Retrieve the list of items saved as list-name
CLEARSELECT	Clear the buffer created by the prior SELECT or GET.LIST
EDIT.LIST list-name	Edit an existing list or add a new one with line editor
DELETE.LIST list-name	Remove the list list-name from the designated directory of lists
COPY.LIST list-name	TO: will prompt and can be a new name or an open parenthesis, filename and name to save to a file. TO: (VOC ABC would save to the VOC file with the name ABC

Advanced use of lists for applications

SELECT is used by the query process to store the primary keys used by the subsequent query or application. Sometimes, though, you want to use SELECT to create lists of fields other than the

primary key.

Using SELECT to create a list of keys to a related file

You've seen that SELECT generates a list of record IDs that can then be used by a follow-up process. Often one field in a file will contain keys to a related file. For example, in our MVTUTOR database, each ORDER has a CUSTID field which is the primary key to the CUSTOMERS file. To specify CUSTID in the list, instruct SELECT to take a field value instead of the primary key with the SAVING keyword. We also just want one CUSTID even though there may be multiple orders for a specific customer; so in this case we specify UNIQUE.

1. We want to select customer information from the ORDERS file. Let's type this and see what happens.

```
SELECT ORDERS SAVING UNIQUE CUSTID  
SAVE.LIST ORDER_CUSTS
```

What is ORDER_CUSTS? It is NOT a list of primary keys for ORDERS.

2. Type this and see what happens.

```
GET.LIST ORDER_CUSTS  
COUNT CUSTOMERS
```

Now you can COUNT the customer records you SELECTed from the ORDERS file, because you saved a different field than the primary key.

3. Type this.

```
GET.LIST ORDER_CUSTS  
LIST CUSTOMERS NAME
```

What is being displayed? What do we know about these customers? The LIST query used the information that was selected in the saved list ORDER_CUSTS.

4. The list of primary keys could be ordered before the SAVE.LIST in the SELECT, or as part of the query after the GET.LIST. Enter the following:

```
GET.LIST ORDER_CUSTS  
LIST CUSTOMERS BY NAME NAME
```

The list is the same, but in a different order. The LIST could also have additional select criteria. Only the primary keys in the ORDER_CUSTS list will be candidates for the LIST.

7.0 Writing a simple program in a PARAGRAPH

In this tutorial you will learn how to use paragraphs in the VOC.

In the previous tutorials you have learned how to list data, edit records and their fields with AE, create files, and populate their dictionaries with interactive fields. Everything you have done so far, has been done by writing sentences that describe the command to be executed. These commands can be stored in the VOC as a paragraph so that you can easily reuse them later. Paragraphs can be as simple

as a single line that stores a command as typed at the command line, or a more sophisticated procedure storage with loops and logical branching.

You can look at a paragraph as a sophisticated macro that has a simple syntax for you to store procedures.

Here are the topics you will learn about and practice in this tutorial:

- Creating simple paragraphs
- Displaying text and comments
- All about prompting
- Processing another VOC item
- Labels and branching with GO

Let's get going with adding your first paragraph to the VOC!

7.1 Simple paragraphs and introduction to prompting

Any command you type at the command prompt, you can store in a paragraph.

1. Type this command at the command prompt

```
LIST CUSTOMERS WITH ZIP = "78750" BY NAME NAME STORE STOREPHONE
```

2. Now use AE and save this as a paragraph named ZIP_LIST in the VOC.

```
AE VOC ZIP_LIST
001 PA
002 LIST CUSTOMERS WITH ZIP = "78750" BY NAME NAME STORE STOREPHONE
```

3. Now execute your first paragraph with:

```
ZIP_LIST
```

Congratulations, you are now a programmer! You have written and executed your first paragraph.

4. The paragraph uses "78750" as a hard coded value in the query, but paragraphs can ask for input with a prompt!

Let's add a prompt for the zip code and see how that works. Use AE to edit the paragraph again.

```
AE VOC ZIP_LIST
```

5. Change the "**78750**" portion of the query string so the paragraph asks for the zip code.

You do this by changing it to **<<Customer Zip Code>>**.

In it's simplest form an inline prompt is **<<Question>>** with Question indicating the text that prompts the user for input.

6. Run your new paragraph by typing its name to list customers in any zip code

```
>ZIP_LIST
Customer Zip Code=78750
```

7. Prompting can be that simple AND... it doesn't need to be in a paragraph! You can prompt inline.

Type the command at the command prompt or if you remember the stack commands, change it in the stack

This works even without being in a paragraph.

```
LIST CUSTOMERS WITH ZIP = <<Customer Zip Code>> BY NAME STORE STOREPHONE
```

7.2 Displaying text and comments

Let's explore the display commands CS and DISPLAY and the asterisk (*) to add comments to make the paragraph more readable. The first command is CS which clears the screen.

1. Insert CS as a new second line in your ZIP_LIST paragraph using the AE editor.

```
AE VOC ZIP_LIST
001 PA
002 CS
003 LIST CUSTOMERS WITH ZIP = <<Customer Zip Code>> BY NAME STORE STOREPHONE
```

2. Now when you run the paragraph, it will clear the screen first.

3. Now, let's insert a DISPLAY line before the LIST. DISPLAY command will display to the screen anything that follows it.

```
001 PA
002 CS
003 DISPLAY Customer report for a specified zip code
004 LIST CUSTOMERS WITH ZIP = <<Customer Zip Code>> BY NAME STORE STOREPHONE
```

4. Run the paragraph again and it will clear the screen and display this line before prompting "Customer Zip Code".

5. Though this is not yet a complex paragraph needing comments, we will insert one explaining what the report does

```
001 PA
002 * Display name, store and phone number for a specified zip code
003 CS
004 DISPLAY Customer report for a specified zip code
005 LIST CUSTOMERS WITH ZIP = <<Customer Zip Code>> BY NAME STORE STOREPHONE
```

These comments are optional, but can help explain logic to someone maintaining the paragraph. Comment lines are lines that start with an asterisk and space.

7.3 All about prompting

Now that we have done a simple prompt, let's look at the syntax of the query prompt.

<<[option,]...text...[,error_checking]>>

When using a prompt in a paragraph the options and error checking can affect how the prompt is processed and what is allowed to be entered at the prompt.

Options

Option	Explanation
no options	Prompts for data, then once entered that same prompt will use the data entered
A	Always ask for input rather than the default as above. In other words, the data entered initially is not saved for later prompts. This becomes more important when a prompted input is used in several procedures or LOOP/REPEAT is used.
F(filename,itemid[,attribute,value,sub-value])	Read input from a file/itemid and optionally a specific attribute, value and subvalue of that record
R	Repeat inputs. The separate inputs will be stored with spaces between. CO FL would be the same as entering CO, then FL, then nothing with the repeat option
Cn	Retrieve input from the command line as the nth parameter. ZIP_LIST 78750 would retrieve 78750 as input. ZIP_LIST alone would retrieve nothing as input
In	Similar to Cn except, if nothing was supplied it would prompt for input. ZIP_LIST 78750 would retrieve 78750 as input, ZIP_LIST alone would prompt for it

Let's change our ZIP_LIST paragraph to work with some of the options

1. R for repeat input allows you to enter multiple entries. Use AE to Add the R option to the prompt syntax.

```
AE VOC ZIP_LIST

005 005 LIST CUSTOMERS WITH ZIP = <<Customer Zip Code>> BY NAME NAME STORE
STOREPHONE
*--: C/<</<<R,
005 LIST CUSTOMERS WITH ZIP = <<R,Customer Zip Code>> BY NAME NAME STORE
STOREPHONE
```

2. Run it again. See how it keeps prompting. You can enter as many zips as you want then just press enter at the prompt to proceed.
3. Now use AE and change it to the I option to use the second argument as an input.

```
LIST CUSTOMERS WITH ZIP = <<I2,Customer Zip Code>> BY NAME NAME STORE
STOREPHONE
```

4. Run your new paragraph and it runs as it did originally.

ZIP_LIST

The difference is that now you can enter the zip code from the command line. Run it again from the command line with the zip code appended to the command.

ZIP_LIST 78750

5. The C2 would be similar to I2 except it will not prompt if nothing is supplied.

Change the VOC ZIP_LIST to see this.

What happens with ZIP_LIST?

How about ZIP_LIST 78750?

6. The last option we will cover is the A option. Using the AE editor, remove the current C2 option from line 005 and add a second query statement to the existing ZIP_LIST resulting in this.

```
005 LIST CUSTOMERS WITH ZIP = <<Customer Zip Code>> BY NAME NAME STORE  
STOREPHONE  
006 LIST CUSTOMERS WITH ZIP = <<Customer Zip Code>> BY STOREPHONE STOREPHONE  
NAME STORE
```

7. Now, when you run the report **ZIP_LIST** it will prompt you with Customer Zip Code and run the first report by name then the second report by phone without prompting. You will have to scroll back the screen to see them.

8. Change line 006 to have <<**A, Customer Zip Code**>> and run it again.

See how it now prompts for each report.

9. Another way to do that without the A option would be to have the prompts differ. Lets do that by using AE on lines 005 and 006 to appear like this.

```
005 LIST CUSTOMERS WITH ZIP = <<BY NAME - Customer Zip Code>> BY NAME NAME  
STORE STOREPHONE  
006 LIST CUSTOMERS WITH ZIP = <<BY PHONE - Customer Zip Code>> BY STOREPHONE  
STOREPHONE NAME STORE
```

Error checking your prompts

Pattern checks

You can add error checks to prompts in your paragraphs. The error checks use pattern matching to only allow input specified by your pattern.

- nA - Number of alpha characters
- nN - Number of numeric characters
- nX - Number of characters
- "foo" - match "foo" literally

The n represents a number from 0 up.

If n is zero (0), it has a special meaning in that it means any number of alpha, numeric or other characters.

Conversion codes

You can also use a conversion code for such things as date and time. This is differentiated from a pattern by being surrounded by parenthesis.

Error checking is best explained by example. Here are a few common uses.

Example use	Prompt form
3N"-4N Phone number 555-1212	<Customer phone number,3N"-4N>>
5N Zip codes in the US	<<Zip code,5N>>
2A State code like FL, AL, CO etc.	<<Enter State,2A>>
"T" OR "F" Allow T for true or F for false	<<Is this True or False,"T" OR "F">>
1A2N1A3N is a pattern match for A12B345 or C33D123, etc	<<Enter Product Number,1A2N1A3N>>
1N1A0A would be a number and anything from 1 to any number of letters	<<Enter division number and letter,1N1A0A>
(D) D is the standard ICONV for dates in BASIC syntax	<<Enter Order date,(D)>>
(MT) MT is the standard ICONV for time in BASIC syntax	<<Enter Order Time,(MT)>>

Now, let's add the correct error checking in the ZIP_LIST paragraph.

1. Edit the ZIP_LIST paragraph with AE.

```
AE V0C ZIP_LIST
```

2. Change the prompts to have **5N** error checking on line **5** and line **6**.

```
C6/>>/,5N>>
```

3. When you run it, it will ask for the zip code as it did. What happens if you put in TX?

Separation of prompt from command

There is an exception to the comment explained in the prior section. Prompts can be specified prior to the statement in a paragraph for better readability.

1. We can change our paragraph to have the prompt prior to the statement, including the options and error checking; then the statement just has the prompt.

```
005 * <<BY NAME - Customer Zip Code,5N>>
006 * <<BY PHONE - Customer Zip Code,5N>>
007 LIST CUSTOMERS WITH ZIP = "<<BY NAME - Customer Zip Code>>" BY NAME NAME
STORE STOREPHONE
008 LIST CUSTOMERS WITH ZIP = "<<BY PHONE - Customer Zip Code>>" BY STOREPHONE
STOREPHONE NAME STORE
```

This would obviously be more pertinent if there were lots of commands with lots of prompts.

7.4 Processing another VOC item or command

Paragraphs, in their simplest form, are running the items in your VOC. This includes everything from the commands provided with your MultiValue application server, such as LIST and SORT, to items you and other users add to the VOC. This includes:

- BASIC programs. Like BLOCK.TERM or other applications developed by software engineers.
- Other paragraphs. One PA item can execute another and upon completion it will return to the PA item that called it.
- Sentences saved in the command stack.
- PROC procedural files

Let's add a new paragraph that runs the ZIP_LIST.

1. Create a new paragraph named ZIPL with

```
AE VOC ZIPL
001 PA
002 ZIP_LIST
```

2. Now run

```
ZIPL
```

This is an example of a paragraph running a paragraph.

3. Now use AE to add **BLOCK.TERM HI** to the bottom of the ZIPL paragraph. This will print HI on the terminal.
4. Execute the ZIPL paragraph with this new line and see what happens when the report is complete or you Quit out of it.

7.5 Labels and branching with GO

A label can be added to allow branching to that label. Labels are added with a colon (:) at the end. The GO command addresses that label without the colon. Labels can either be all numeric or alphanumeric, but alphanumeric labels must start with a letter, not a number.

1. Let's add a label to our ZIPL paragraph and a simple GO to that label.

Change this...	to this...
001 PA 002 ZIP_LIST 003 BLOCK.TERM HI	001 PA 002 GO HELLO 003 ZIP_LIST 004 HELLO: 005 BLOCK.TERM HI

Now when you run the ZIPL paragraph, it will skip the ZIP_LIST and go to the HELLO portion directly.

This example would seldom be used in a non-tutorial setting, but labels and GO when combined with IF...THEN add lots of flexibility to a paragraph.

We will be visiting that in the next session.

7.6 IF ... THEN logic and labels

In this section you can practice adding IF-THEN logic to a paragraph in combination with labels.

Your MultiValue expert Dan shows you in this helpful video how he uses this logic in a simple paragraph to look up all or specific orders for a given customer.

[Adding IF...THEN logic to a paragraph](#)

7.7 LOOP ... REPEAT logic

This type of logic is familiar to programmers and allows a process to run a loop of instructions over and over. It is included with paragraph, but has no exit logic normally included for WHILE or UNTIL something happens. LOOP ... REPEAT logic can only be used with a IF ... THEN which allows for the paragraph to exit the loop.

We can add this logic to our VOC ORD to allow it to repeat until we ask it to exit. We can add a LOOP at the top and REPEAT at the bottom and then it will repeat forever. The problem is, we need to be able to exit when done.

Watch this video to see how Dan solves this problem

[LOOP repeat logic in a paragraph](#)

7.8 Clearing input data with CLEARPROMPTS

The CLEARPROMPTS command does essentially what the A option does, but it does it for all prompts at once. You can use it to simplify the VOC ORD.

See how Dan adds it to his paragraph in this video.

[CLEARPROMPTS statement](#)

7.9 Stacking data with DATA command

DATA is the complement to the <<prompt>> in that you can feed data to any input. This will usually be used when running a MultiValue command or an application from a paragraph that ask for input.

Using LIST.LABEL command to create label format to the screen will prompt for parameters.

Instead of remembering and keying the label parameters each time, it can be in the paragraph with a DATA statement.

1. Execute the command `LIST.LABEL CUSTOMERS BY NAME NAME STORE STOREPHONE`
2. For the parameters enter the following:

```
COUNT ,ROWS ,SKIP, INDENT, SIZE, SPACE {,C} ? 3,5,0,0,26,0
```

Q out of the display

3. Now, with a paragraph, we could have it auto fill the label parameters with a DATA statement

```
AE VOC 3UP
001 PA
002 * <<Enter label file and up to 5 fields>>
003 LIST.LABEL <<Enter label file and up to 5 fields>>
004 DATA 3,5,0,0,26,0
```

4. Execute the new paragraph and enter the query without having to remember the parameters.

3UP

Enter label file and up to 5 fields= CUSTOMERS BY NAME NAME STORE STOREPHONE

8.0 Increasing Performance with Hashed Files and Indexes

In the third tutorial you learned that hashed files are much much quicker than directory files and we never really told you why, or how they worked. Now that you know how to query data and how data is stored in files, you can investigate how hashed files work and use them to optimize files.

When working with hashed files you generally follow this process:

- Analyze the file to be resized or created for the average record size and number of records.
- Calculate an approximate modulo and separation.
- Create a new file or resize an existing one with the new modulo and separation.
- Evaluate the results.

In this tutorial we offer a little bit of theory, followed by a step-by-step guide and a challenge to be solved in the MVTUTOR account.

In this video Dan explains how hashed files work.

[How hashed files work](#)

Storing records in files

Every flavor of Multi-Value application server comes with their own set of hashing algorithms to store records into files. These algorithms allow the application server to optimally store and search the data you are manipulating. Over the last few decades the available algorithms have expanded quite a bit and diverge among the database you select. UniVerse offers many different types of hashed files dependent mostly on the type of key. However, all of the same principles still apply and in this tutorial you'll look more into these principles.

Average record size

Items have a size in bytes. Most characters take 1 byte unless they are non-standard ASCII which may be defined with multiple bytes.

The size of an item is determined by:

- The data
- The overhead for a record
- The key

The **data** of an item represents character storage size of your data literally.

The **overhead** of a record is determined by the application server you use, for UniVerse this is about 30 bytes per record.

The **key** of a record is also stored with the item. In the MVTUTOR account most keys consist of a four character string representing a number. So the size of a key in this case is 4 bytes.

8.1 Number of access points: frames

It is important to know how many items you expect to store and manipulate in the file. A hashed file contains a number of frames in which items in the file will be physically stored. The number of frames are set when creating or resizing a file. It's not important to get this correct when you first create your file as you can always resize it as your application grows. The number of frames assigned in your file is determined by **modulo**.

[7 frames allocated](#)

A file with a modulo of 7 would have these frames allocated (see image above). Items get added to those frames based on a file-type, which determines the algorithm that calculates where to put the items, spreading them evenly among the frames. As the frames fill, an application server may add more frames underneath each modulo frame. Each modulo frame serves as the entry point to that group of frames. This is how multi-valued files allow for unlimited growth. Your estimated modulo frames, or "entry points", is the modulo and the closer that modulo is to the actual groups needed, the more efficient the file will be for storing, reading, searching, and editing records.

When determining the modulo to use, it is an industry standard to take the nearest prime of an approximated modulo.

Overflowing

[Those frames in overflow](#)

Perhaps you are starting to store more data than can be allocated for in the 7 original frames. The original 7 frames (modulo) set up for the file will end up overflowing and the 7 frames may grow as illustrated above. Now groups 1, 4 and 6 are taking 2 frames each. The first frame is the modulo frame or base frame and the other is an overflow frame and together we call them a group.

If the original modulo was 7 and there are 700 frames worth of items to be stored, then this could mean 100 or more extra reads are needed to access an item in a file. Though multi-value files will not fail when sized too small, you can see how the performance can be greatly impacted when a file is undersized.

Optimizing the type of hashed files

Universe allows for several different file types which determine the hashing algorithm that is used to distribute items over the frames of the file. In very large applications that need to ensure the quickest access to items, an administrator can use tools to precisely choose which file type to use for each individual file created. However, the default file type which is extremely fast and will work for most files is a type 18. This course will be using that default and focuses on the modulo and separation to determine the number and size of frames that are originally allocated to a file based on the data to be stored within them.

Frame size: Separation

The size of each frame in bytes is also important. When you request or store a record, the hashing algorithm decides which frame to read or write based on its primary key. The industry standard reads and writes data in chunks of 4096 bytes. You can observe this in the earlier tape drives from the past up to the modern Network File Systems (NFS) of today. To make best use of this hardware standard and optimize reading and writing speed and size of your application server, you can use a frame size of 4096 bytes. Even when thinking of BIG DATA in the terabytes, it is still read and written in 4096 byte size chunks.

Separation is what determines the size of a frame because it multiplies the application servers' base block size to get to the frame size.

$\text{framesize} = \text{separation} * \text{blocksize}$

UniVerse has a block size of 512 bytes so usually a separation of 8 works best.

This assumes that items in your file generally are smaller than 1024 bytes. When your items are bigger you can optimize for larger frame sizes. If the records are typically larger than that, it is usually best to have a frame size at least 4 times that of the normal item.

The reason we're thinking so small in a world of terabytes like today, is that individual items are still very tiny in a database.

8.1 How to create optimal hashed files

To summarize what we've just covered:

- You work with the average size of a record and number of records you expect your file to have.
- You approximate the modulo to assign the original number of frames that are assigned in the file.
- You determine the size of each frame with by multiplying the base block size with separation. Generally aiming for a frame size of 4096 bytes.

Creating optimal hashed files is oftentimes referred to as "more of an art than a science". Still, we'll use some basic math to get close to the art we want to help you set up your files optimally.

$$\text{modulo approx} \sim (\text{numrecords} * (\text{recordsize} + \text{overhead} + \text{keysize})) / \text{framesize}$$

8.2 Let's put this into practice by creating a file

In the MVTUTOR account we want to start processing our bank transactions, to help process receivables and bookkeeping. We're going to create a new file to import our transactions and want to set it up optimally. We're going to add the bank transactions to the account in a file called TRANSACTIONS.

The process we'll follow is:

- Analyze or estimate the average record size and number of records.
- Calculate an approximate modulo and separation.
- Create a new file.
- Evaluate the results.

Analyze

1. We estimate each transaction will average 90 bytes in data. In the data we store the following information and expect the average size of that data in a record.

DATE	IBAN	IBAN	AMOUNT	DESCRIPTION	CODE	total
5	18	18	5	25	16	87 bytes (approx 90)

2. The overhead for UniVerse is 30 bytes.
3. Each key will consist of 10 characters.
4. We process about 100 bank transactions every single day.

Q: How many records will there be for a whole year (365 days)?

$$100 * 365 = 36500$$

5. With the given size, what is the correct frame size?

The normal frame size of 4096 will work fine.

Calculate

6. What will be our optimal modulo on UniVerse?

$$(36500 * (90 + 30 + 10)) / 4096 \sim 1158$$

The nearest prime is 1163, optimal modulo for one year of transaction processing.

Create

7. Create the file, Execute:

1. For UniVerse **PICK**

```
CREATE.FILE TRANSACTIONS 1,2,3 1163,8,18
```

2. For Universe **PRIME**

```
CREATE.FILE TRANSACTIONS 18 1163 8
```

Evaluate

8. Evaluate the results. Import last years transactions with

```
IMPORTTRANSACTIONS
```

9. Run

```
FILE.STAT TRANSACTIONS
```

8.3 Let's experience what happens when a file overflows

We're having a wildly successful year and are processing way more transactions than we could ever have anticipated.

1. Run the program OVERLOADTRANSACTIONS to load more transactions.

```
OVERLOADTRANSACTIONS
```

2. How full are the groups when you show them with FILE.STAT?

```
FILE.STAT TRANSACTIONS
```

FILE.STAT now shows mostly 200% full.

There are 136,500 records, with an average of 116 bytes.

Notice that the file still works when you LIST or SORT. It'll also work fine when adding or editing records.

3. What is the correct modulo?

$$(136500 * (116 + 30 + 10)) / 4096 \sim 5198$$

Nearest prime number: 5209

4. How to resize the file?

Warning: When you are resizing a file, it locks the entire file. Your application cannot access the data inside until the resize is complete. Typically you resize during maintenance hours or use a file system that allows concurrency.

Do not try this on a live production system

Do try this in the MVTUTOR training account

5. Resize the file with the resize verb

```
RESIZE TRANSACTIONS 18 5209 8
```

6. Check the statistics on the file after:

```
FILE.STAT TRANSACTIONS
```

8.4 Creating Indexes

Why use secondary indexes?

All hashed files are indexed. Hashed files have a primary key and can instantly be accessed by that key. An image of this would be a very well organized file cabinet. For example, the CUSTOMERS file has several Utotem Convenience locations, including the one with primary key 8888. You could go to a

filing cabinet that had the files in numeric order and could quickly find the file in the 8000 drawer, after 8887 and before 8889.

On a multi-value server, you can instantly access this information with an application or data edit utility. This is done by reading the item with the primary key or index in that same manner. It would take a long time to go through all the numeric cabinets if you wanted to access all the customers with the name Utotem Convenience, since you would have to go through all the drawers and all the files and select them.

On the MV server, you would SELECT those items by the DICT item NAME. This will read all the customers and create a list of those you specified. With a small file, that is a simple task that takes milliseconds; but if this file contained the web customers for a large business with millions of customers and you wanted all those with NAME = "BOB JONES" the task is no longer simple and does not process quickly. This is when a secondary-index or alternative index is necessary to give immediate access to a list of items without having to read all the items in the file.

This is analogous to having a list updated for BOB JONES containing all the primary keys. The file or cabinet is in numeric order, but your list can instantly return the numbers that are needed for BOB JONES using the list. The overhead of keeping up this list is very minimal and a select of items from thousands or even millions of items becomes nearly instantaneous. A simple test of this with 10 million customers with a secondary index on NAME returned a list of them in less than 2 seconds. The same query without the index took anywhere from 20 to 70 or more seconds. So, SPEED is why.

What should and should not have a secondary index?

Fields with unique values that need to be processed quickly or are used often, for example name, phone or possibly city. Although there is little overhead for a secondary index, you do not want to have too many that would burden everyday updates. Each time a record is altered, the secondary indexes for that record are automatically updated. There is also no advantage to an index on fields such as gender or yes/no fields where the index would not aid in the selection.

Who uses secondary indexes?

Anyone who wishes to select and access files quicker.

Secondary indexes are part of a well-designed application and many applications use special commands within BASIC to speed processing.

The multi-value server query language will also use these indexes, so everyday reporting and queries will benefit from the speed and once the indexes are initially built, they will maintain and update themselves without any need for user intervention.

8.5 How are they created , built and listed?

There are two steps to creating a secondary index. The first step is defining it and the second step is building it. Once an index is built, the updating of the index is automatic and requires nothing more.

Creating an index or indexes with CREATE.INDEX filename index DICT Indexes can be created with a simple command and the DICT item you want to index. Remember, LIST DICT filename will give you the list of fields.

Warning: Like resizing, when you are indexing a file, it locks the entire file. Your application cannot access the data inside until the indexing is complete.

Do try this on your MVTUTOR account.

1. Let's create a couple indexes on the CUSTOMERS FILE

```
CREATE INDEX CUSTOMERS NAME
```

```
CREATE INDEX CUSTOMERS ZIP
```

2. Build the indexes

```
BUILD INDEX CUSTOMERS ALL
```

3. List the indexes for the CUSTOMERS file

```
LIST INDEX CUSTOMERS ALL
```

Finished!

You have reached the end in this course.